Attention Is All You Need

Almog Schwartzberg
Shlomi Hazan
Amit Herner



How Chatbots Help Us Through "Understanding" Text?

Answer Questions

We ask them questions, they answer

Translate Languages

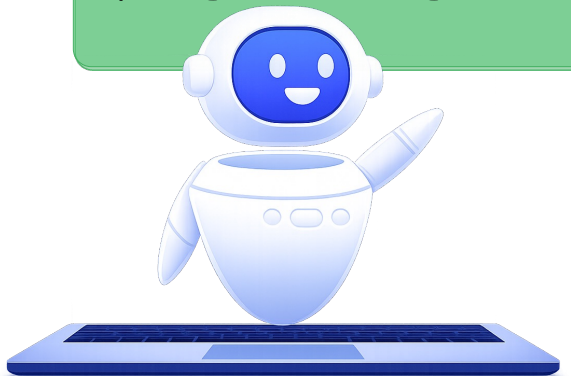
Convert text between languages

Summarize Documents

Sum up long articles into short summaries

Write & Debug Code

Generate and fix programming code



Explain Concepts

Break down complex ideas

Maintain Context

Track meaning across conversations

Sequence Transduction

(התמרת רצפים)

Sequence transduction refers to any task where an

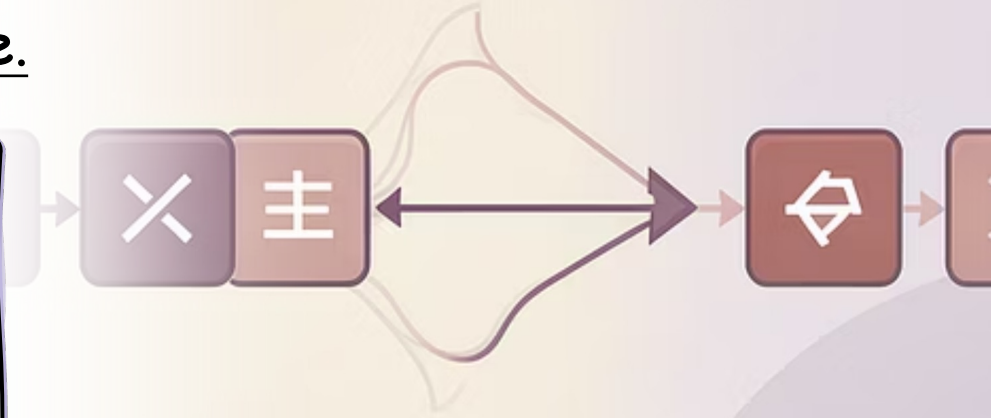
input sequence is transformed into an output sequence.

Summarization

A long article is condensed into a short summary

Translation

An English sentence becomes a German sentence.



Question Answering

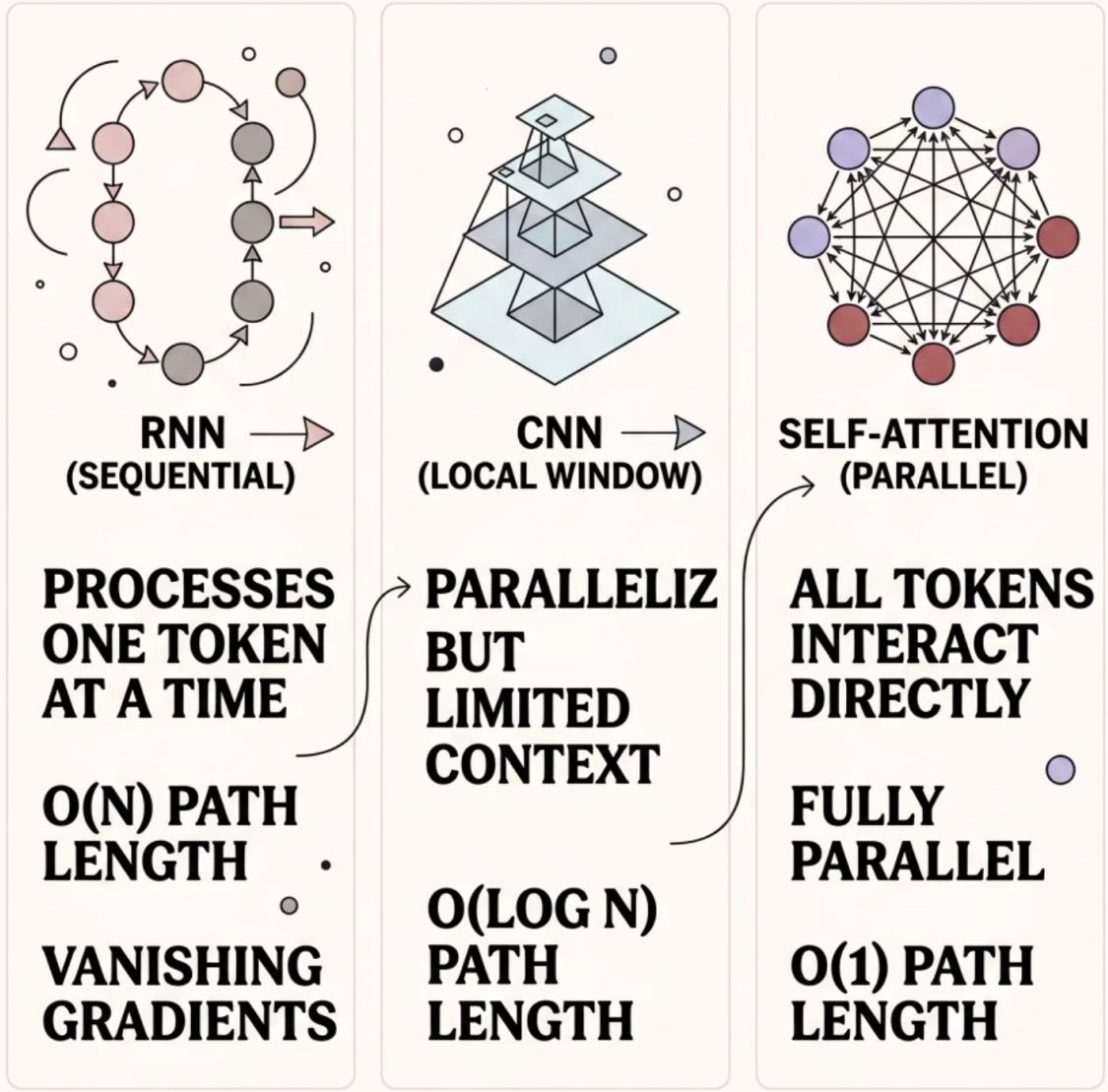
A question (sequence of words) is transformed into an answer (sequence of words).

Part 1 — The World Before Revolution

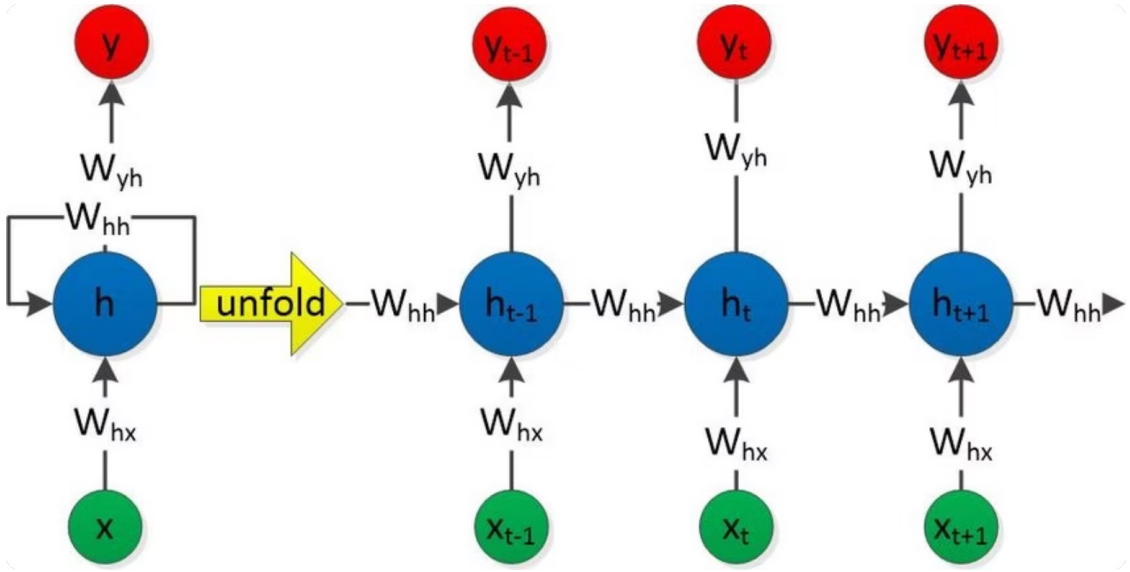
Why did sequence modeling need a new architecture?

Before the Revolution: How Were Sequences Processed?

Before 2017, sequence models relied on 3 main approaches — each with fundamental limitations for language understanding.



RNN - Recurrent Neural Network



How RNNs Work?

Hidden state updated at each step:

$$h_t = f(h_{t-1}, x_t)$$

Where x_t = input token, h_{t-1} = previous hidden state, f = learned transformation.

Minimum $O(n)$ sequential operations.

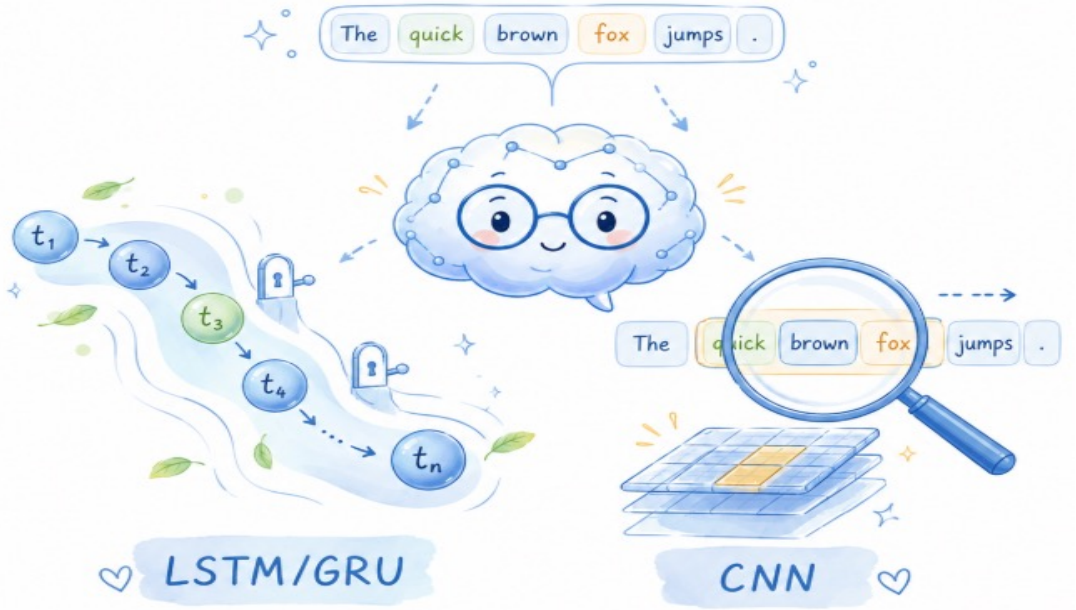
ⓘ "The **apples** in the basket on the table **are** red" — "apples" and "are" are distantly connected, but the signal weakens across many steps.

LSTM / GRU & CNN

LSTM / GRU: Gated Memory

Uses gates to control what to **remember**, **forget**, and **pass forward**. Better at preserving long-range information — **but still process tokens sequentially.**

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$$



CNN - Convolutional Neural Network: Local Windows

Kernel size 3 on **"The quick brown fox jumps"**:

> [The quick brown] → [quick brown fox] → [brown fox jumps]

Once we choose an option, we neglect the others and eventually "miss" distant words (quick+fox).

More parallelizable than RNNs, but distant dependencies require many layers. They can't see the whole picture immediately.

Part 2 — The Motivation Behind the Paper

From sequential processing to global attention

Why Was a New Model Needed?

Parallelization
(מקביליות)

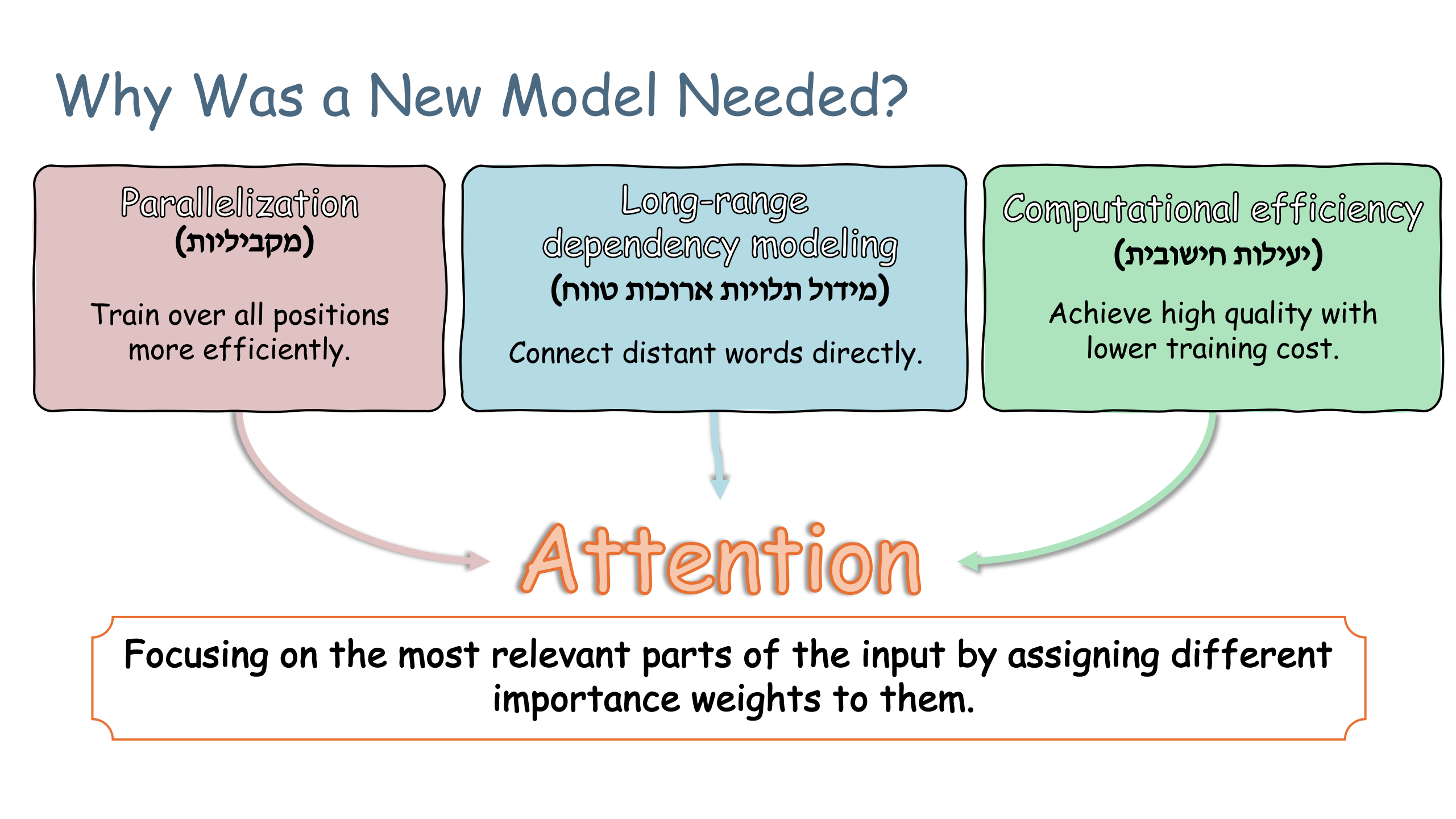
Train over all positions more efficiently.

Long-range dependency modeling
(מידול תלויות ארוכות טווח)

Connect distant words directly.

Computational efficiency
(יעילות חישובית)

Achieve high quality with lower training cost.



Focusing on the most relevant parts of the input by assigning different importance weights to them.

Example

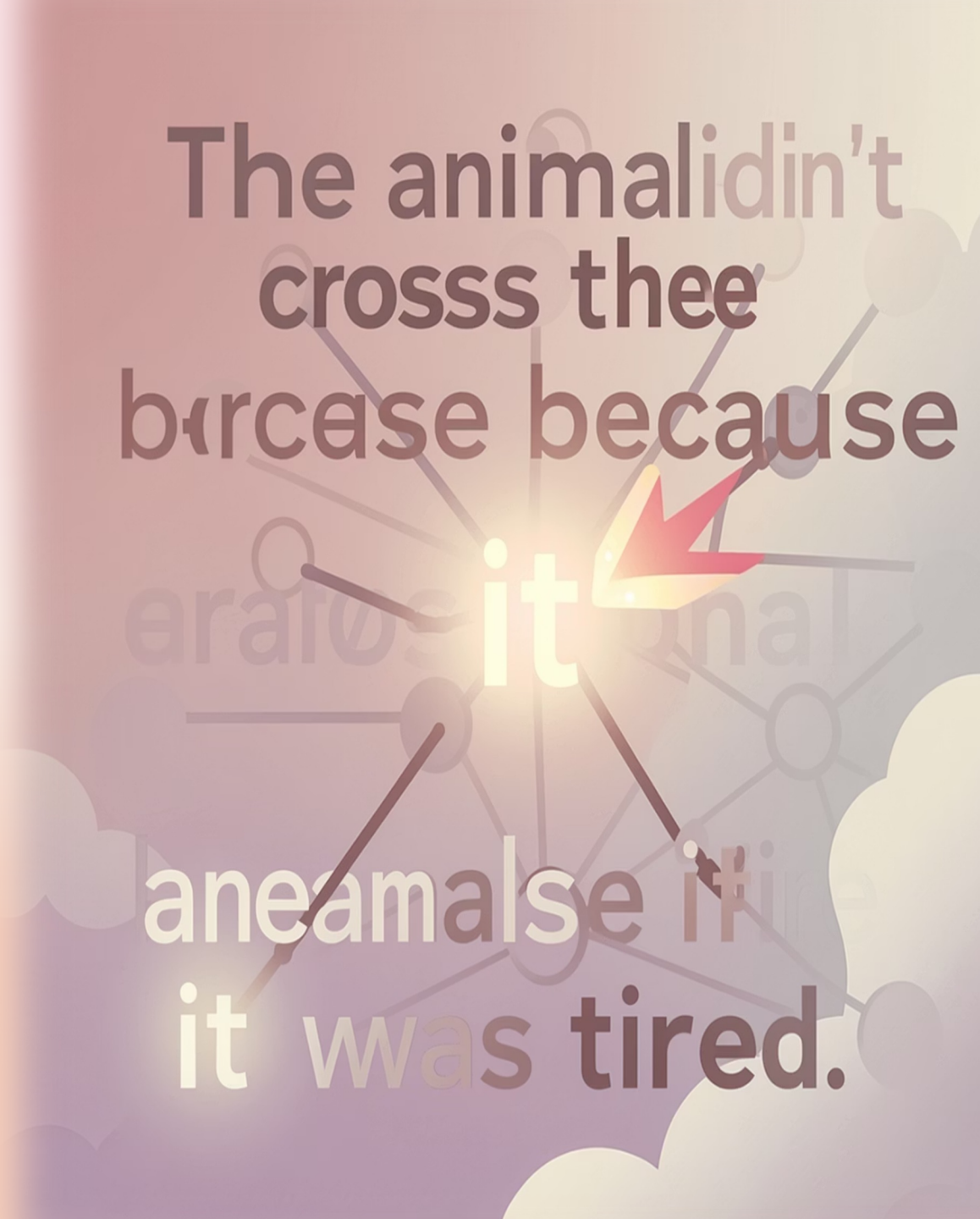
Consider the following prompt:

> I spilled coffee all over my laptop.
>
> Now it won't turn on. What should I do?

For the token `it`, the self-attention mechanism might assign:

High attention to `laptop`, indicating `it` refers to the laptop.

Lower attention to `coffee`, as `it` is unlikely to refer to the coffee.

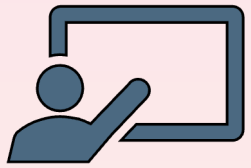




Part 3 — The 2017 Paper

A new architecture was born

The Researchers



Ashish Vaswani
Google Brain

Noam Shazeer
Google Brain ✡️

Niki Parmar
Google Research

Jakob Uszkoreit
Google Research

Llion Jones
Google Research

Aidan N. Gomez
University of Toronto / Google Brain

Łukasz Kaiser
Google Brain

Illia Polosukhin
Google Research

The Research Question:

Can we build a Sequence Transduction model based entirely on attention?

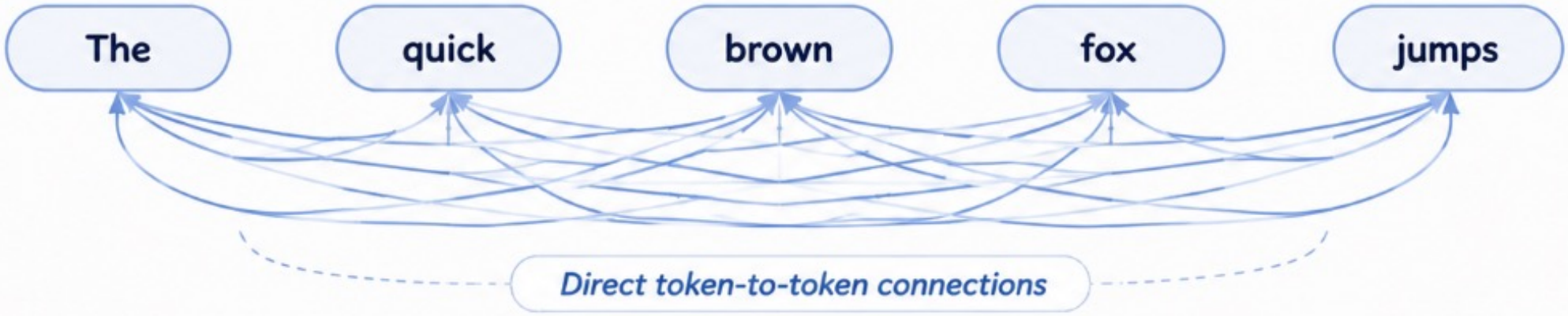


- **No RNN**: Eliminating sequential processing.
- **No CNN**: Overcome limitations of long-range dependencies.
- **Sequence Transduction**: Maintaining the ability to translate inputs to outputs.
- **Efficient training**: Achieving high performance with parallelizable computation.
- **Quality**: Matching or exceeding State of the art (הכי טוב בשוק).



Hypothesis: Attention Is Enough

If every token can **directly** attend to every other token,
sequential computation may not be necessary.
Fundamentally changing how models process sequences.



$$x_i \leftrightarrow x_j \qquad \forall i, j \in \{1,2,\ldots,n\}$$

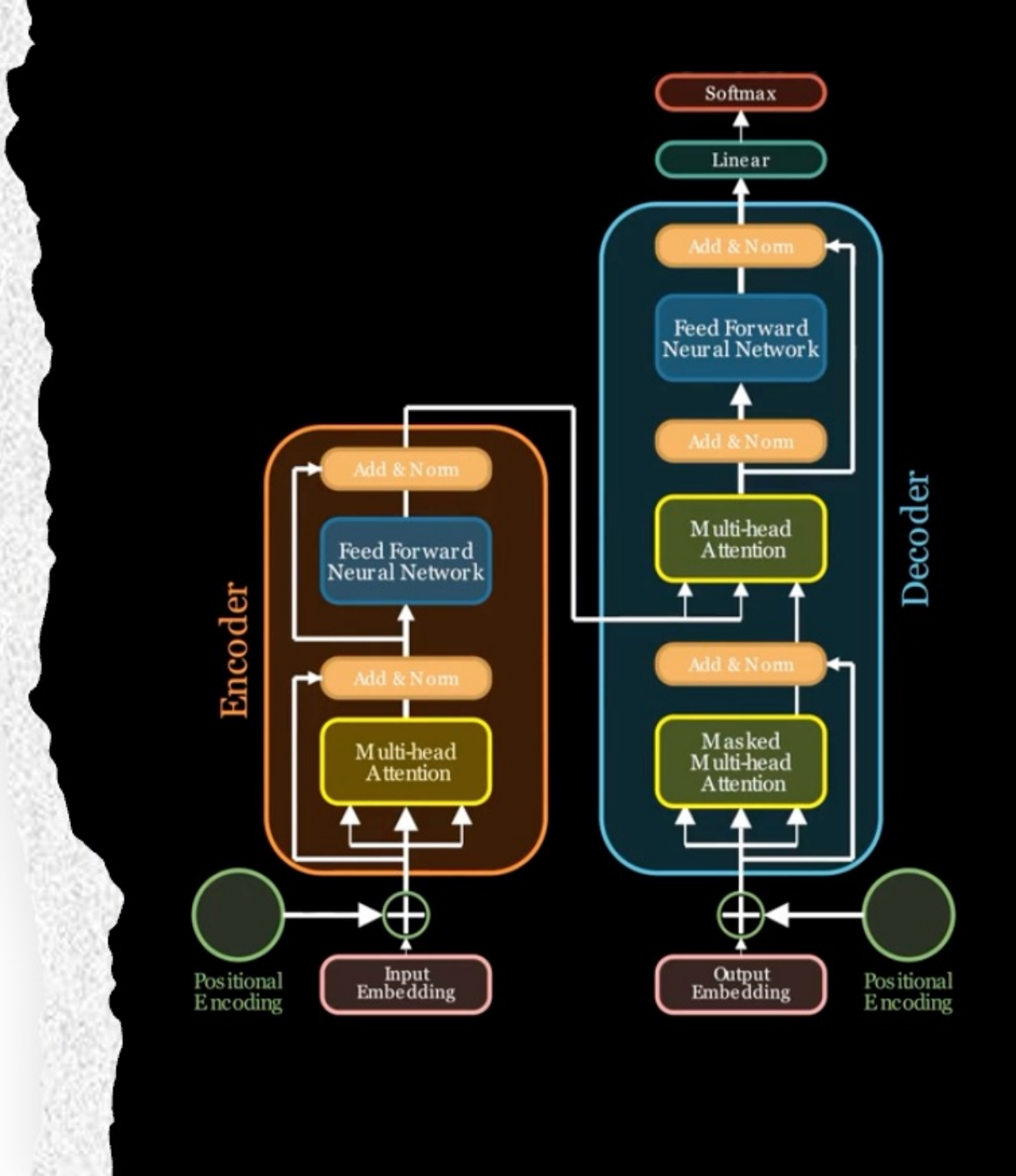
Maximum Path Length: O(1)

Any token can reach any other token in one step.

Part 4 — The Transformer Architecture

How attention became the center of the model

The Solution: Transformer



Vectorization

1

Tokenization with Byte-Pair Encoding (BPE)

The sentence is split into tokens — words or subword units.

`unbelievable` → `un` + `believe` + `able`

2

Token ID

Each token receives a numerical ID from the vocabulary.

`believe` → `5321`
`able` → `918`

3

Input Embedding

Each token ID is mapped to a vector. In the original Transformer base model, $d_{model} = 512$.

`believe` → `[0.21, -0.44, 0.08, ..., 0.13]`

4

Positional Encoding

Because the Transformer has no recurrence or convolution, it needs explicit information about word order.

Attention: The Core Mechanism

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{1}{\sqrt{d_k}} \cdot QK^T\right) \cdot V$$

Q (Query)
What I'm looking for.

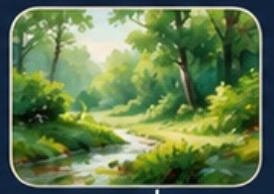
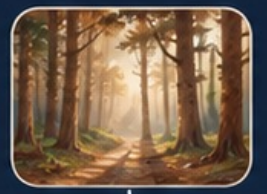
K (Key)
What each token offers.

V (Value)
The token's content

$\sqrt{d_k}$
Scaling factor

Example: token embeddings



a	fluffy	blue	creature	roamed	the	verdant	forest
$\vec{E}_1$	$\vec{E}_2$	$\vec{E}_3$	$\vec{E}_4$	$\vec{E}_5$	$\vec{E}_6$	$\vec{E}_7$	$\vec{E}_8$

The Encoder – Transform Embeddings to Attention

Each token in the input sequence is transformed into 3 distinct vector representations: **Query** ($\vec{Q}$), **Key** ($\vec{K}$), and **Value** ($\vec{V}$).

Query

$$\vec{Q} = W_Q \cdot \vec{E}$$

what the token is looking for

Key

$$\vec{K} = W_K \cdot \vec{E}$$

what the token offers

Value

$$\vec{V} = W_V \cdot \vec{E}$$

the token's content

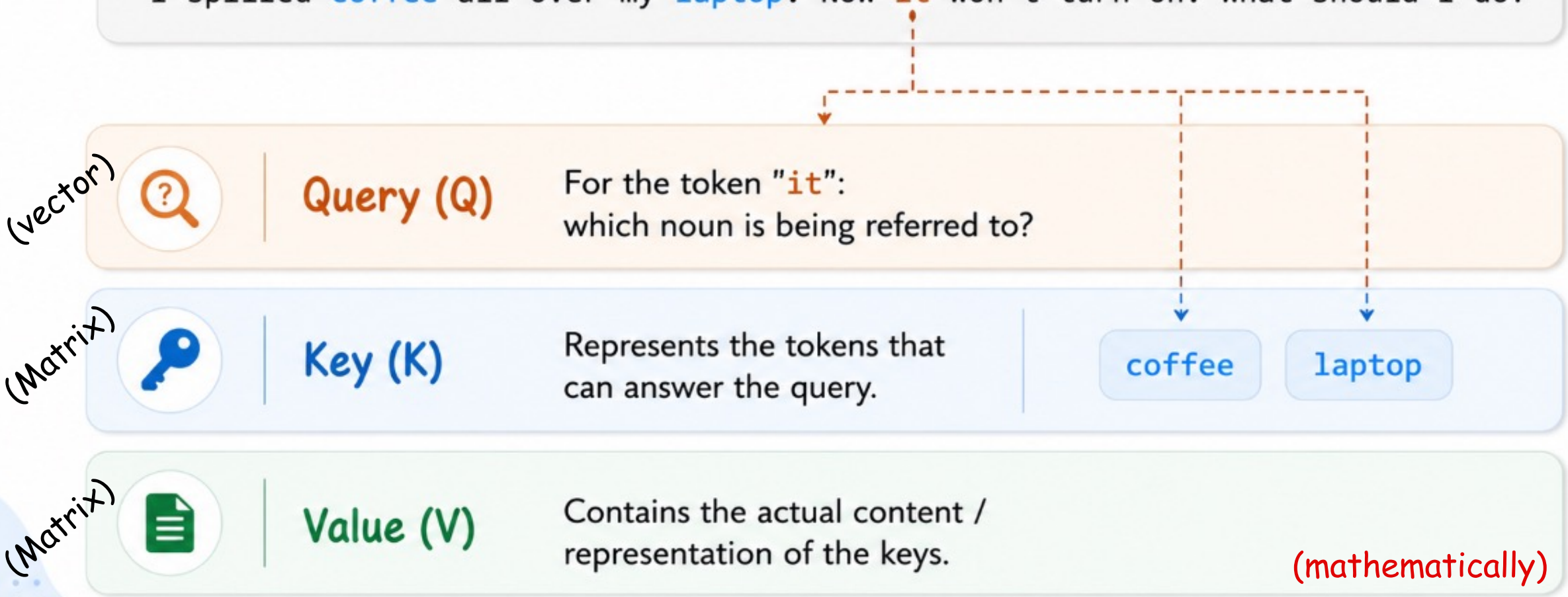
- E : the input word embedding vector
- W_Q, W_K, W_V : learned weight matrices that project the embedding into task-specific subspaces

During training, the W_Q, W_K, and W_V matrices start with random values and are updated by **backpropagation**, allowing the model to capture meaningful relationships between tokens.

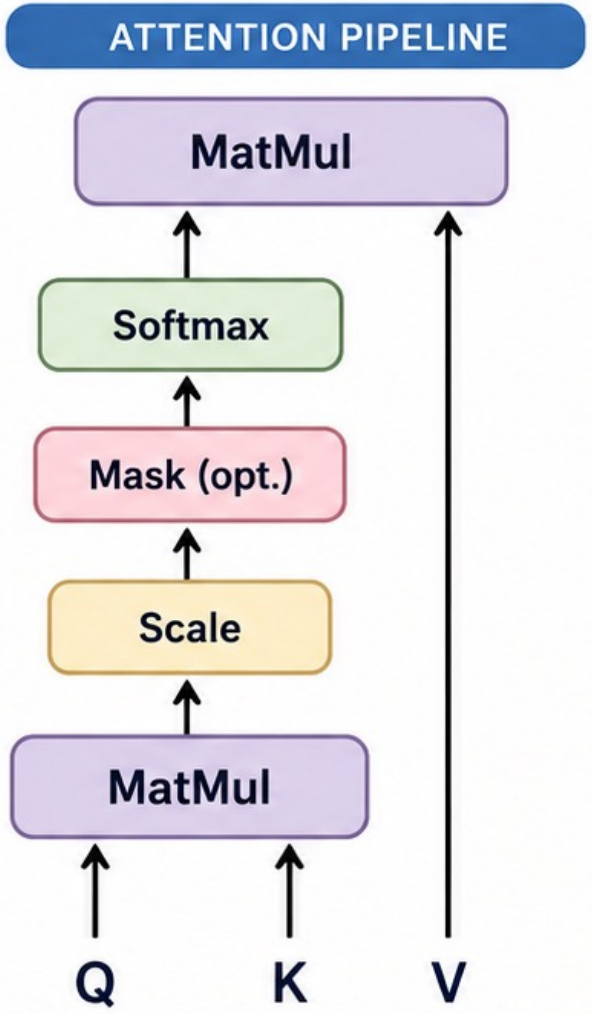
Example

Consider the prompt:

I spilled coffee all over my laptop. Now it won't turn on. What should I do?



Scaled Dot-Product Attention



$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{1}{\sqrt{d_k}} \cdot QK^T\right) \cdot V$$

$$\downarrow$$

Attention for "it" =

$$= \text{softmax}\left(\frac{1}{\sqrt{d_k}} \cdot Q_{it} \cdot K^T\right) \cdot V =$$

$$= \ldots = \mathbf{0.9} \cdot V_{\text{laptop}} + \mathbf{0.1} \cdot V_{\text{coffee}}$$

Multi-Head Attention: Multiple Views at Once

Single Head → **Multi-Head**



Anaphora Resolution
(זיהוי כינוי גוף)

Anaphora is a word (like *it*, *he*, *she*...) that refers to a noun mentioned earlier. This head solves a specific puzzle: "Who is this pronoun talking about?"



Semantic Nuances
(קשרים סמנטיים ואסוציאציות רחבות)

Analyze the surrounding words and real-world context to clarify the exact meaning of a term. For example, for "Harry', the model can use clues like "wizard' or "Prince" to decide which specific "Harry" the text is talking about.

Heads operate in parallel.

Final step - Add & Norm

$$\vec{x}_{new} = \text{LayerNorm}\left(\vec{x}_{orig} + \text{Sublayer}(\vec{x}_{orig})\right)$$

— What each term means —

$\vec{x}_{new}$

The final output vector that combines the token's original meaning with its actual context.

LayerNorm

Normalization layer that keeps the values balanced and stable.

$\vec{x}_{orig}$

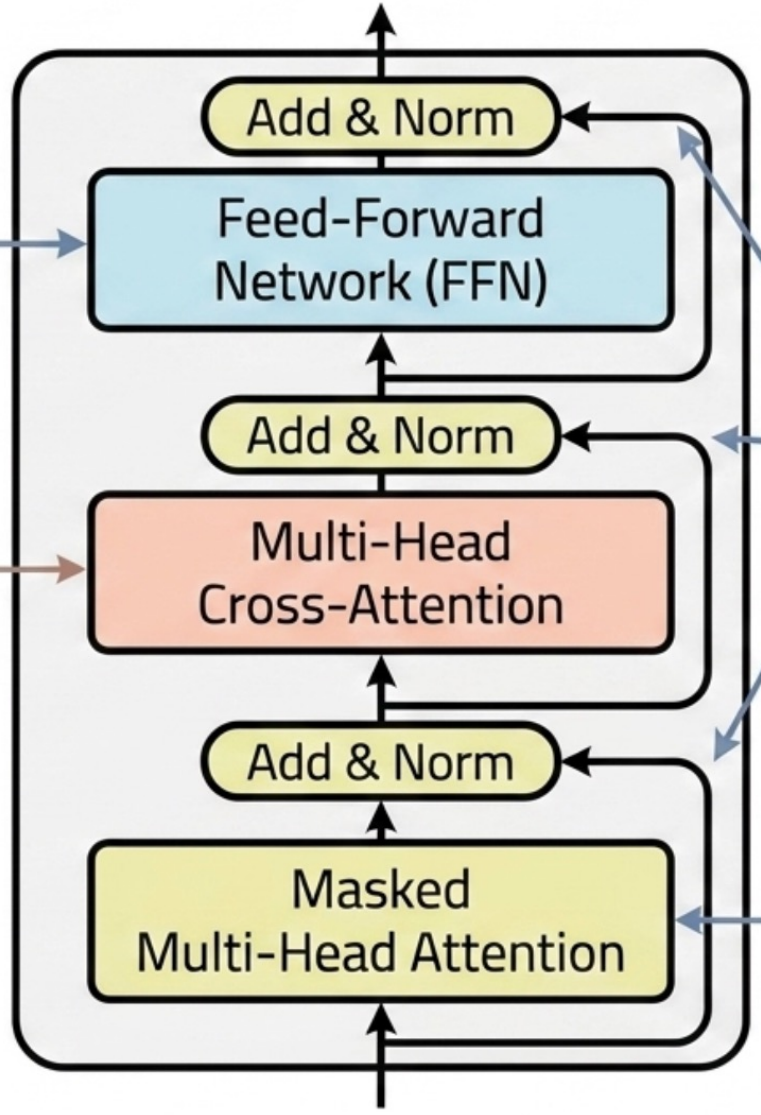
The original vector (word embedding).

$\text{Sublayer}(\vec{x}_{orig})$

The multi-head attention layer output that adds contextual information.

The encoder's task was to obtain all the $\vec{x}_{new}$ vectors

The Decoder – Generate Answers to Our Questions



Individual processing and refinement.

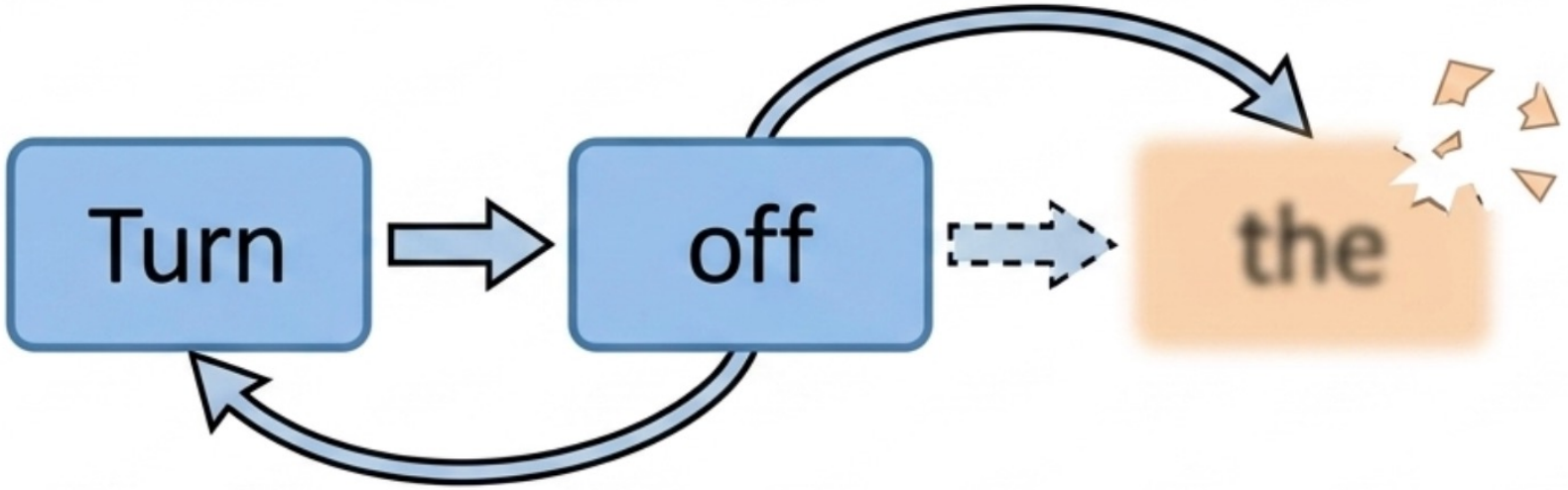
Extracting relevant information from the Encoder.

Preventing information loss and normalizing the data between layers.

Searching for context in the already generated text.

Masked Self Attention – Peeking Prohibition

The **Decoder** is required to predict the next word. To learn correctly (or generate correctly), it must **not see future** words.



The word 'off' can only look for context in the words 'Turn' and 'off'. The following words are completely hidden.

Built-in Causality – A token at position i can only attend to positions less than or equal to i.

The Mechanism of Masking

Calculating Attention Scores +
Replacing Future with Negative Infinity

2.3	-∞	-∞	-∞	-∞
1.1	2.1	-∞	-∞	-∞
0.9	-0.5	0.9	-∞	-∞
1.5	0.1	3.2	2.7	-∞
3.2	0.1	2.7	1.8	-1.2

Softmax

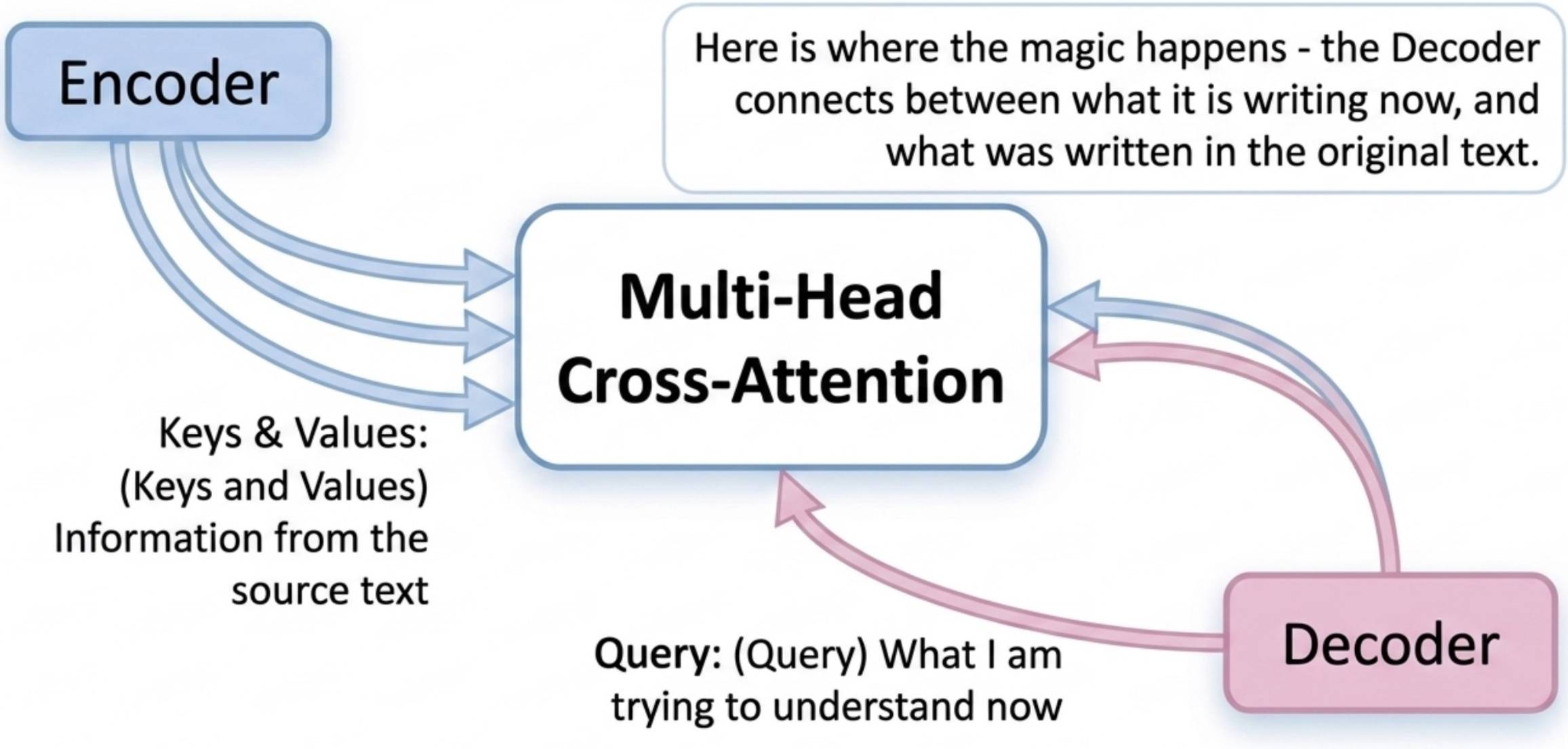


Converting Negative Infinity
to Absolute Zero

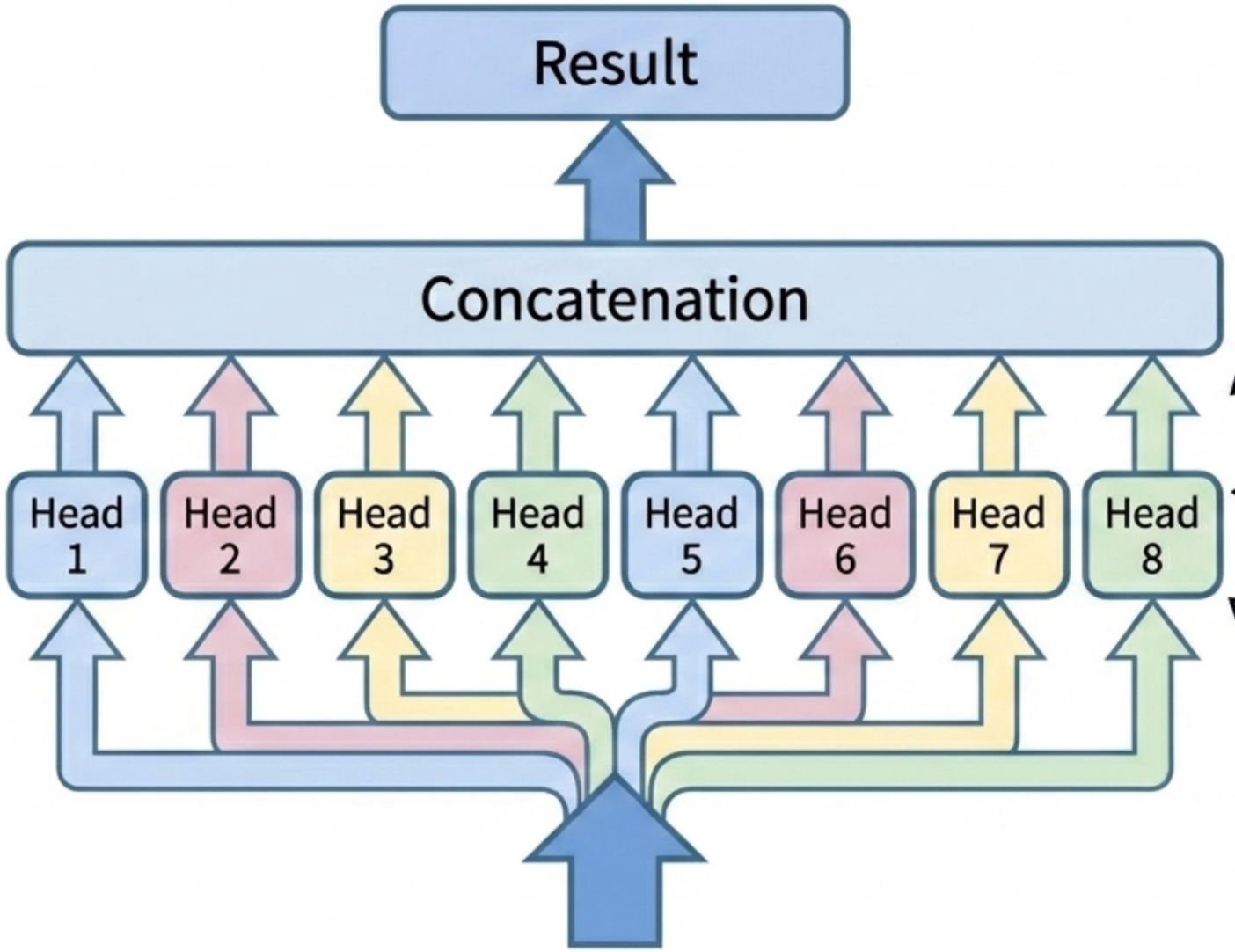
0.8	0	0	0	0
0.2	0.0	0	0	0
0.5	0.0	0.5	0	0
0.3	0.2	0.6	0.1	0
0.2	0.6	0.1	0.1	0.7

Complete disconnect between the current word and words that have not yet been created, without changing the parallel architecture.

The Encoder-Decoder Attention



Someone said Multi-Head?

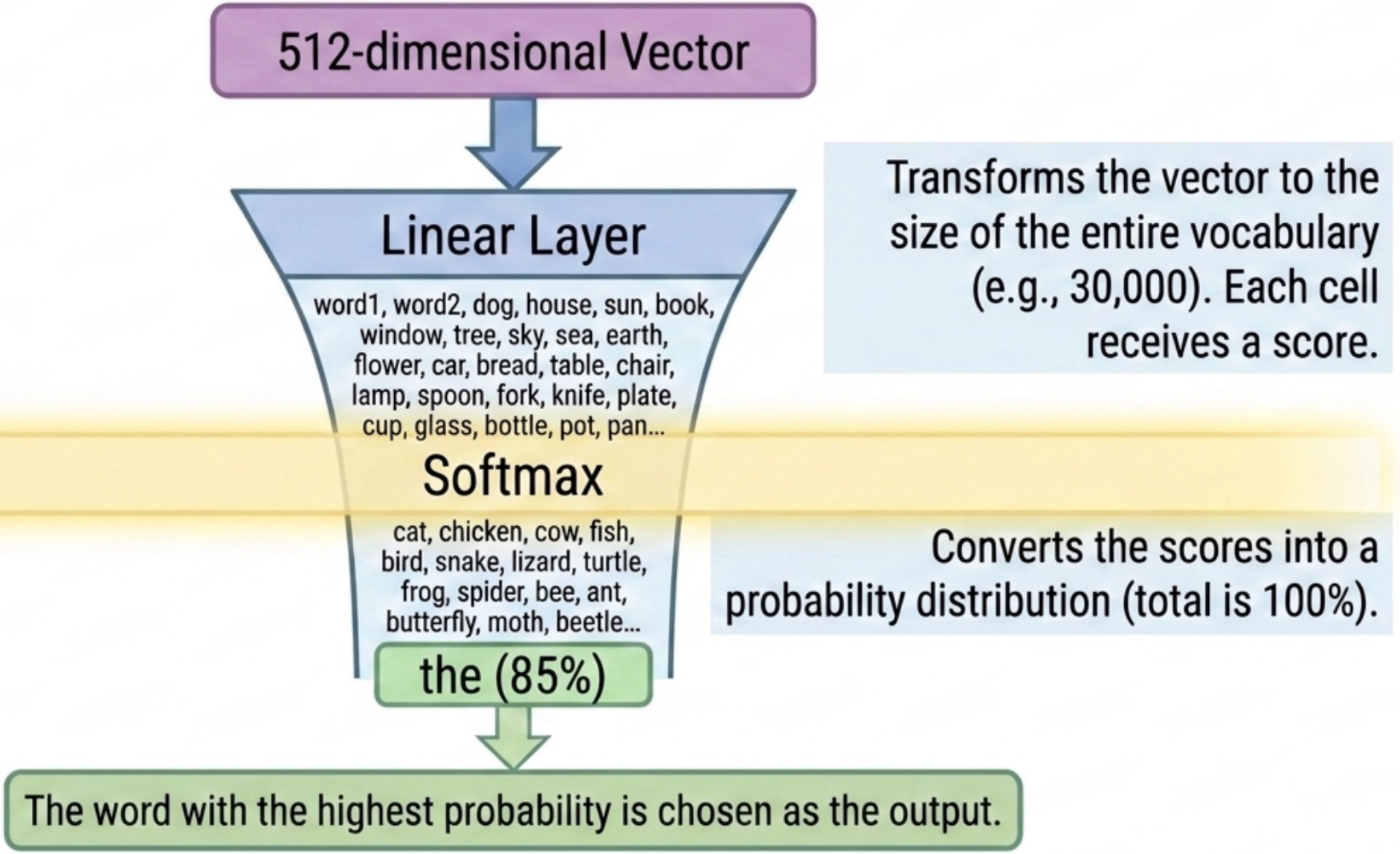


1. Instead of looking for one type of context, attention is split into 8 heads.

2. One head focuses on grammar, a second on sentiment, and a third on tense or context.

3. All heads compute conclusions in parallel, and the results are concatenated back into a single rich vector.

Generating The Next Word (Output)



Part 5 — Original Training Details

How the Transformer was trained and evaluated

Training Data

English → German

Dataset: WMT 2014 English-German

- Approximately 4.5 million sentence pairs (English & German)
- Shared source-target vocabulary of about 37,000 tokens

English → French

Dataset: WMT 2014 English-French

- Approximately 36 million sentence pairs (English & French)
- 32,000 word-piece vocabulary

Translation benchmarks used in training

Training Hardware and Schedule

Transformer models were trained using **8 NVIDIA P100 GPUs**, showcasing the power of parallel processing for deep learning at the time.

Base Transformer

- Training Steps: **100,000**
- Approximate Duration: **12 hours**
- Step Time: **~0.4 seconds**

Big Transformer

- Training Steps: **300,000**
- Approximate Duration: **3.5 days**
- Step Time: **~1 second**

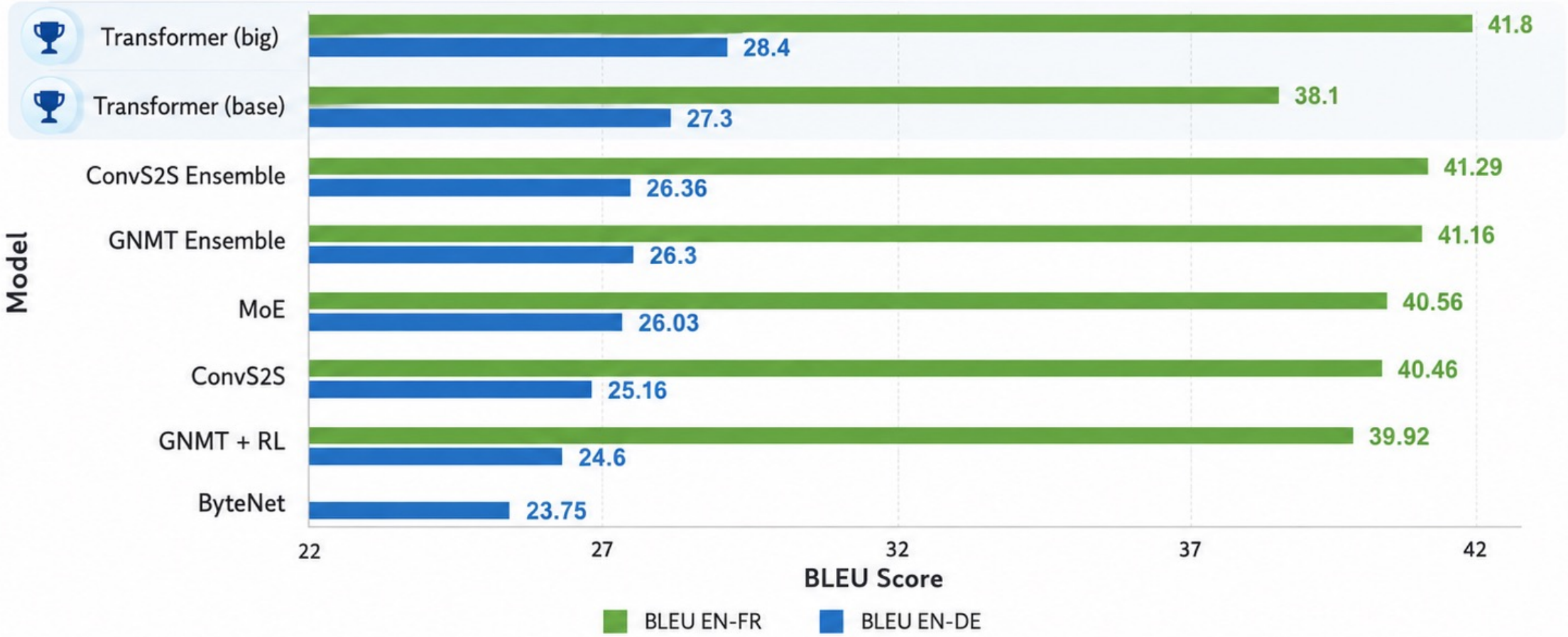
The Transformer achieves strong results with **significantly less training time** compared to many previous sequential models.

Part 6 — Results and Evaluation

How well did the Transformer perform?

Results: BLEU score

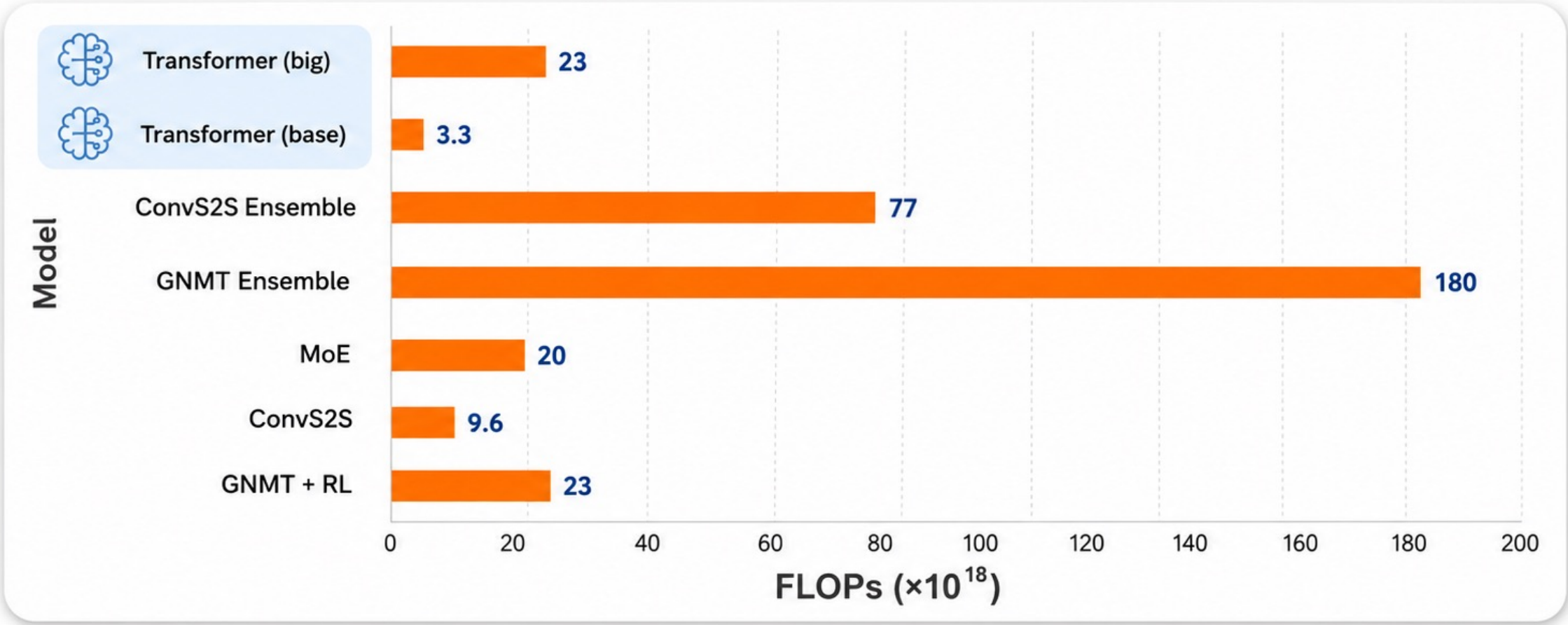
We measure the 'accuracy' with BLEU score: built specifically for tasks like machine translation.



Transformer models achieve the strongest BLEU scores, especially on EN-DE, while remaining highly competitive on EN-FR.

Results: Training Cost

We measure the training cost using FLOPS - The amount of mathematical operations required for training.



Transformer models achieve strong performance with significantly lower training cost than several earlier baselines.

Part 7 — Why Is the Transformer Efficient?

Parallelization, path length, and global context

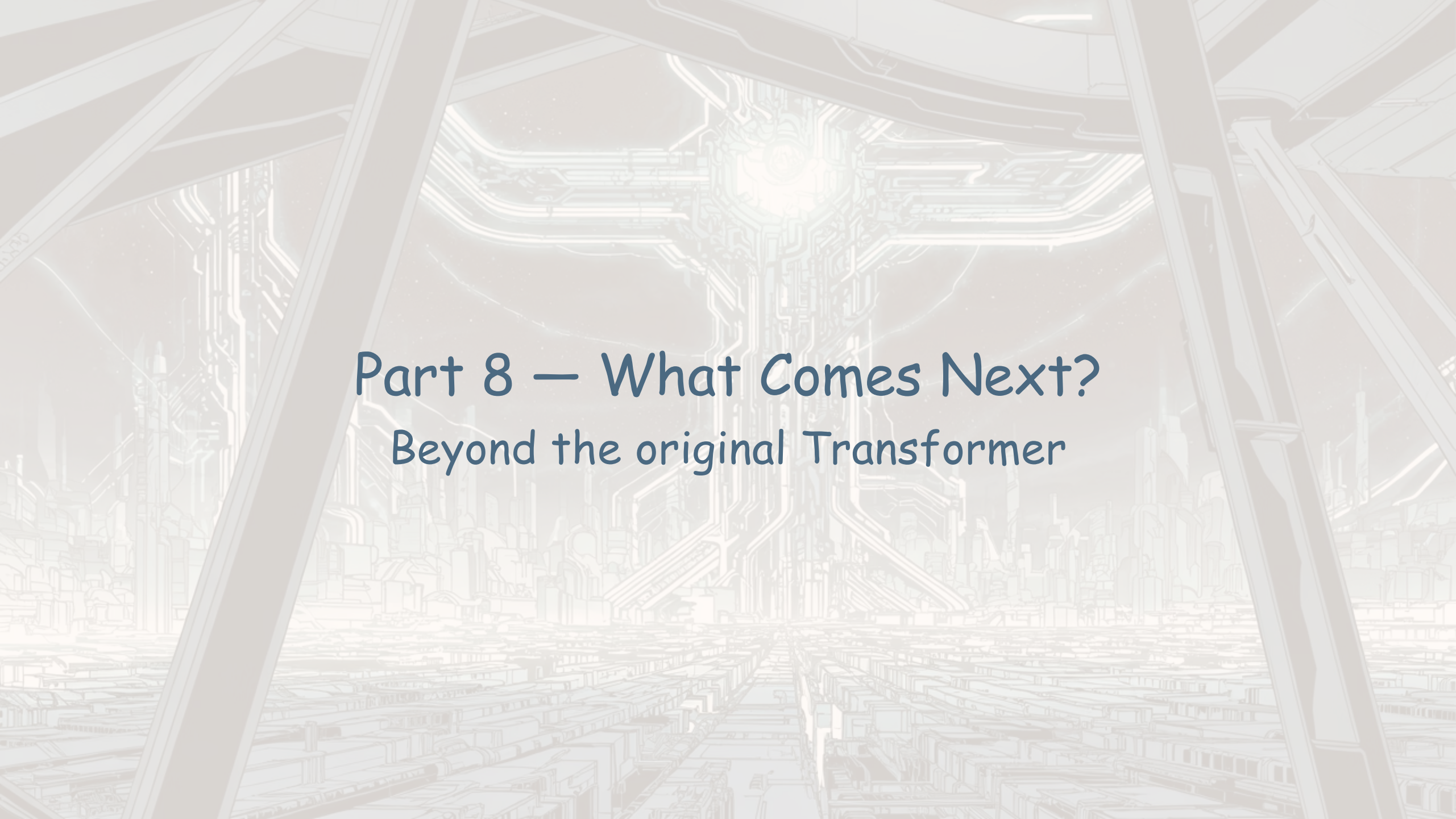
Complexity

Model Type	Time Complexity	Sequential Ops	Max Path Length
★ **Self-Attention**	$O(n^2 \cdot d)$	$O(1)$	$O(1)$
RNN	$O(n \cdot d^2)$	$O(n)$	$O(n)$
CNN	$O(k \cdot n \cdot d^2)$	$O(1)$	$O(\log_k n)$

- $\boldsymbol{n}$ = sequence length (number of tokens in the input)
- $\boldsymbol{d}$ = representation dimension (size of vector embeddings)
- $\boldsymbol{k}$ = convolution kernel size (for CNNs)

Self-attention demonstrates particular efficiency under certain conditions.
It is most efficient when $n < d$.

This comparison highlights that while self-attention has a quadratic dependency on sequence length (*n^2*), it excels in having constant sequential operations and maximum path length. This makes it highly parallelizable, a critical advantage for training on modern hardware, especially for tasks like sentence-level translation where *n* is often smaller than *d*.



Part 8 — What Comes Next?

Beyond the original Transformer

What Comes After Transformer?

The original "Attention Is All You Need" paper didn't just introduce a groundbreaking architecture; it also laid the groundwork for numerous future research avenues. Many of the challenges and opportunities hinted at in 2017 have become central to modern AI development.

- Do language models truly understand, or do they only predict?
- Will the Transformer remain the dominant architecture for AI?
- Is full attention sustainable and efficient for very long sequences?
- How should we balance performance, cost, and ethical responsibility in AI development?
- What might be the next major breakthrough after attention?

Open questions for the future of AI